

DAA Assignment 1

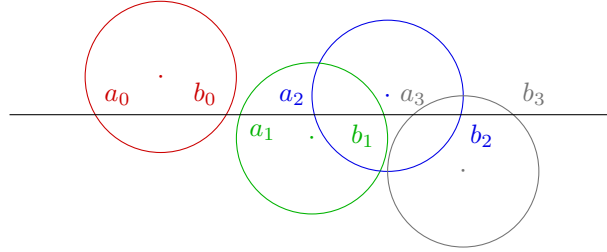
Abhiram Aravind [Sr No 25238]

1 KE Question 5

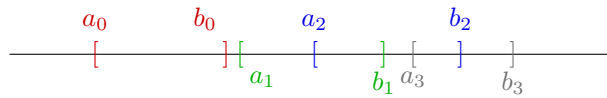
Input: Locations of n houses.

Output: The minimal set of base stations such that each house is within 4 miles of at least one base station.

The problem requires that each house be covered by a base station within 4 miles of it. So, if we draw circles of radius 4 miles centred at each house, then there must be at least one base station within each circle.

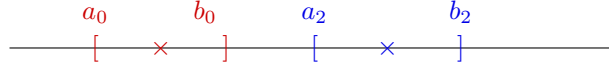


Let $[a_0, b_0], [a_1, b_1] \dots [a_{n-1}, b_{n-1}]$ be the intervals formed on the road by the intersecting circles. We see that since base stations must be built on the road, each interval must contain at least one base station.



Lemma 1.1. *If there are at most k non-overlapping ranges, then at least k base stations are needed to cover the houses.*

One can see that since none of the intervals overlap, they cannot share base stations, hence each interval contains a different base station. This puts a lower bound on an optimum solution to the problem. If there exists a set of k non-overlapping intervals, then an optimum solution must have at least k base stations.



Lemma 1.2. For an interval $[a, b]$, if $[a_0, b_0], [a_1, b_1] \cdots [a_{k-1}, b_{k-1}]$ are the intervals that overlap with it and $\forall i < k, b \leq b_i$ (That is, $[a, b]$ has the least upper bound), then the intervals $[a, b], [a_0, b_0], [a_1, b_1] \cdots [a_{k-1}, b_{k-1}]$ pairwise overlap with each other.

Proof. Suppose there exists two intervals $[a_p, b_p]$ and $[a_q, b_q]$ that do not overlap. Assume $b_p < a_q$. Since $[a, b]$ has the least upper bound, then $b \leq b_q < a_q \implies [a_q, b_q]$ does not overlap with $[a, b]$, a contradiction. Which means all the intervals that overlap with $[a, b]$ must also pairwise overlap with each other. $\square$

Lemma 1.3. If there are a set of finite intervals $[a_0, b_0], [a_1, b_1] \cdots [a_{n-1}, b_{n-1}]$ that pairwise overlap (That is, $[a_i, b_i] \cap [a_j, b_j] \neq \emptyset \forall i, j < n$), then there exists at least one point that is common to all intervals.

Proof. One can state for all i, j :

$$[a_i, b_i] \cap [a_j, b_j] \neq \emptyset \implies a_i \leq b_j \wedge a_j \leq b_i$$

$$\implies \max(a_i, a_j) \leq \min(b_i, b_j)$$

$$\implies \max_{k < n} (a_k) \leq \min_{k < n} (b_k)$$

$$\text{If } \max_{k < n} (a_i) \leq p \leq \min_{k < n} (b_i)$$

$$\text{Then } a_k \leq p \leq b_k \quad \forall k < n$$

$\implies$ For all intervals, the point p is common to them. $\square$

We now come to the algorithm that solves the base station problem optimally:

1. For each house, find the interval on the road within 4 miles from it.
2. Sort the intervals in ascending order by their upper bound.
3. Use the greedy Interval Selection Algorithm discussed in class to come up with the maximum set of non-overlapping intervals S .
4. For each interval I_i in S , come up with a point that is common to all the intervals $I_{i+1}, I_{i+2} \cdots I_{i+k}$ that overlap with it. (Any point between the maximum lower bound and minimum upper bound of $I_i, I_{i+1}, I_{i+2} \cdots I_{i+k}$ should work)
5. The points thus produced are where the base stations ought to be placed.

Proof of optimality: At each step of the Interval Selection Algorithm, the algorithm picks the interval with the least upper bound. By lemma 1.2, every interval that overlaps with it overlaps with each other. Lemma 1.3 ensures there is a guaranteed point common to all the overlapping intervals. Thus, we have one base station for each interval in S , giving us a total of $|S|$ base stations. Since S is a maximum set of non-overlapping intervals, by lemma 1.1, an optimal solution cannot have fewer base stations than $|S|$. $\square$

Runtime Complexity:

- Constructing intervals: $O(n)$
- Sorting: $O(n \log(n))$
- Interval Selection Algorithm: $O(n)$
- Placing points: $O(n)$
- *Total:* $O(n \log(n))$

2 KE Question 17

Input: Set of intervals $\mathbb{I}$ on a circle.

Output: The maximum set of non-overlapping intervals $\mathbb{S}$.

Constraint: Time complexity polynomial in n .

Let $\mathbb{C}_i$ be the set of all intervals that overlap with interval $I_i \in \mathbb{I}$, including I_i itself. Let $\mathbb{S}_i$ be a maximum set of non-overlapping intervals in $\mathbb{I} \setminus \mathbb{C}_i$.

The following algorithm gives the optimal solution to the problem:

1. Pick an interval $I_0 \in \mathbb{I}$.
2. With I_0 as the first interval, sort the intervals of $\mathbb{I}$ in ascending order of their upper bounds.
3. For all $I_i \in \mathbb{C}_0$, calculate $\mathbb{S}_i$ using the greedy Interval Selection Algorithm discussed in class.
4. $\mathbb{S} := \{I_k\} \cup \mathbb{S}_k$, where $k = \operatorname{argmax}_j |\mathbb{S}_j| \ \forall j \text{ s.t. } I_j \in \mathbb{C}_0$

Proof of optimality: We see that:

- An optimal solution must contain at least one of the intervals in $\mathbb{C}_0$. If it didn't, then one can add I_0 to the optimal solution, violating optimality.
- In $\mathbb{I} \setminus \mathbb{C}_i$, one can easily establish a first interval and a last interval that do not overlap with I_i . Thus, the Interval Selection Algorithm correctly produces $\mathbb{S}_i$.
- $\{I_k\} \cup \mathbb{S}_k$ is the maximum set of non-overlapping intervals in $\mathbb{I}$ where we are forced to pick I_k . Thus, by picking k for which $\mathbb{S}_k$ is largest, we are guaranteed an optimal solution.

□

Worst case runtime complexity:

- Sorting: $O(n \log(n))$
- Interval Selection Algorithm for one interval: $O(n)$
- Performing Interval Selection Algorithm for each interval in $\mathbb{C}_0$: $O(n^2)$
- Total: $O(n^2)$

3 JE Question 16

Input: Two unsorted arrays S and W of sizes m and n .

Output: Weighted mean x .

Constraint: Time complexity $O(n)$

We can easily adopt the QuickSelect with Median of Medians approach discussed in class to come up with a linear time algorithm to compute the weighted median. Define `WeightedMedian(l, r)` as the algorithm:

1. Choose a pivot p in $S[l \dots r]$ using the Median of Medians approach described in class.
2. Partition $S[l \dots r]$ about p . Make sure to swap corresponding elements in W while swapping elements in S .
3. Calculate $\Sigma W := \text{sum of elements of } W$, $\Sigma W_{<p} := \text{the weights of elements in } W \text{ with index less than } p$ and $\Sigma W_{>p} := \text{the weights of elements in } W \text{ with index greater than } p$.
 - If $\Sigma W_{<p} \leq \Sigma W \wedge \Sigma W_{>p} \leq \Sigma W$, output p ;
 - If $\Sigma W_{<p} \leq \Sigma W$ but $\Sigma W_{>p} > \Sigma W$, output `WeightedMedian(p+1, r)`;
 - If $\Sigma W_{>p} \leq \Sigma W$ but $\Sigma W_{<p} > \Sigma W$, output `WeightedMedian(l, p-1)`;

Then `WeightedMedian(1, n)` gives the required weighted mean.

Proof of correctness: At every step after partition:

- If $\Sigma W_{<p} \leq \Sigma W \wedge \Sigma W_{>p} \leq \Sigma W$, then we have found out the required weighted mean.
- If $\Sigma W_{<p} \leq \Sigma W$ but $\Sigma W_{>p} > \Sigma W$, then that means we have gone too far left, and a value $> S[p]$ is what we need. Hence, we recurse in the right sub-array.
- If $\Sigma W_{>p} \leq \Sigma W$ but $\Sigma W_{<p} > \Sigma W$, then that means we have gone too far right, and a value $< S[p]$ is what we need. Hence, we recurse in the left sub-array.

□

Time complexity: Given we can compute the Median of Medians in $O(n)$ with the guarantee that the resulting pivot p is within $\frac{3n}{10}$ and $\frac{7n}{10}$, we have the worst case complexity:

$$T(n) = T\left(\frac{7n}{10}\right) + cn$$

$$T(n) \in O(n)$$

4 JE Question 21 (a)

Input: Two sorted arrays A and B of size n .

Output: The median of $\text{sort}(A \cup B)$.

Constraint: Time complexity $\Theta(\log n)$

Lemma 4.1. *Say we have an increasing sequence S , with median m . Consider two disjoint sub-sequences S_0 and S_1 of S with medians m_0 and m_1 respectively. If $m_0 < m_1$, then $m_0 \leq m \leq m_1$.*

Proof. Suppose S_0 and S_1 have lengths l_0 and l_1 respectively. If $m_0 < m_1$, then we see that there are $\lfloor \frac{l_0}{2} \rfloor$ elements less than m_0 in S_0 and $\lfloor \frac{l_1}{2} \rfloor$ elements less than m_1 in S_1 . Since $m_0 < m_1$, then there cannot be more than $\lfloor \frac{l_0}{2} \rfloor + \lfloor \frac{l_1}{2} \rfloor$ elements less than m_0 in S .

Suppose $m < m_0$. Then there cannot be more than $\lfloor \frac{l_0}{2} \rfloor + \lfloor \frac{l_1}{2} \rfloor - 1$ elements less than m . $\lfloor \frac{l_0}{2} \rfloor + \lfloor \frac{l_1}{2} \rfloor \leq \lfloor \frac{l_0+l_1}{2} \rfloor \implies$ There cannot be more than $\lfloor \frac{l_0+l_1}{2} \rfloor - 1$ elements less than m in S , which contradicts the fact that m is the median of S . Therefore $m_0 \leq m$.

Similarly, one can show $m \leq m_1$. Therefore $m_0 \leq m \leq m_1$. $\square$

We design the function $\text{med}(p, q, l)$ that does the following:

- If $l = 1$, then it returns the median of $A[p]$ and $B[q]$.
- Else it calculates $m_a := \text{median}$ of the sub-array of A starting at index p and of length l ; and $m_b := \text{median}$ of the sub-array of B starting at index q and of length l .
 - If $m_a < m_b$, it returns $\text{med}(p + \lfloor \frac{l}{2} \rfloor, q, \lceil \frac{l}{2} \rceil)$
 - If $m_a > m_b$, it returns $\text{med}(p, q + \lfloor \frac{l}{2} \rfloor, \lceil \frac{l}{2} \rceil)$

We note $\text{med}(0, 0, n)$ has time complexity $\Theta(\log n)$.

$$T(n) = T(\lceil \frac{n}{2} \rceil) + c$$

$$\implies T(n) \in \Theta(\log n)$$

Claim 4.2. $\text{med}(0, 0, n)$ returns the median of $\text{sort}(A \cup B)$.

Proof. Assume without loss of generality that at the first step $m_a < m_b$. By Lemma 4.1 $m_a \leq \text{median}(\text{sort}(A \cup B)) \leq m_b$. Since we eliminate the elements less than m_a in A , and those greater than m_b in B , we end up eliminating elements on either side of $\text{median}(\text{sort}(A \cup B))$. We notice that the number of elements less than m_a in A is the same as the number of elements greater than m_b in B , therefore the median of the sorted union of the resulting arrays must be the same as $\text{median}(\text{sort}(A \cup B))$. $\square$

5 JE Question 21 (b)

Input: Two sorted arrays A and B of sizes m and n .

Output: k th smallest element in $A \cup B$.

Constraint: Time complexity $\Theta(\log(m + n))$

Claim 5.1. *For two arrays P and Q of length k each, the k th smallest element of $P \cup Q$ is the lower median of $\text{sort}(P \cup Q)$. (Lower median means the smaller of the two central elements.)*

The claim above is obvious. For a sequence of length $2k$, the k th smallest element is the lower median. If P and Q are sorted, we can find the median in $O(\log k)$ using the algorithm mentioned in the previous question.

For the given problem, it suffices to resize A and B to size k in such a way that the lower median of $\text{sort}(A \cup B)$ is the k th smallest element we are looking for.

Definition 5.1. *We define $\text{resize}(P[1 \dots n], k)$ as a function on $[1 \dots k]$ where:*

- $\text{resize}(P[1 \dots n], k)(i) = A[i]$ where $i \leq n$
- $\text{resize}(P[1 \dots n], k)(i) = +\infty$ where $i > n$

Essentially, when we $\text{resize}(P[1 \dots n], k)$, we create a ‘virtual array’ that is P truncated or padded with $+\infty$ on the right to make it size k .

Claim 5.2. *The median of $\text{sort}(\text{resize}(A, k) \cup \text{resize}(B, k))$ gives the k th smallest element of $A \cup B$.*

Proof. Suppose w is the k th smallest element. As long as we can ensure that the number of elements less than w remains the same after resizing, our algorithm outputs the correct answer. We analyse the three possible scenarios before resizing:

- $m, n \geq k$. We see that $w \leq A[k]$ and $w \leq B[k]$, therefore truncation does not affect any elements less than w .
- $m < k \wedge n \geq k$. We see that $w \leq B[k]$, therefore B can be safely truncated. Since $\text{resize}(A, k)$ is padded with $+\infty$, the number of elements less than w are guaranteed to not increase. A similar argument goes for $n < k \wedge m \geq k$.
- $m, n < k$. Since both $\text{resize}(A, k)$ and $\text{resize}(B, k)$ are padded with $+\infty$, the number of elements less than w remain unchanged.

□

Time complexity: resize has constant access time. Therefore, the time complexity is the time complexity of the median algorithm $= \Theta(\log k) = \Theta(\log(m + n))$. (As k can be at most $m + n$)